xlog: A Universal GPU-Native Engine for Neurosymbolic Integration

The xlog Project
v0.9.2 Technical Whitepaper

June 2026

Abstract

We present xlog, a universal GPU-native engine for neurosymbolic integration: it couples neural perception with the full spectrum of symbolic reasoning—deterministic Datalog, probabilistic inference, and epistemic reasoning over world views—on a single CUDA runtime, keeping all semantic state device-resident with zero host–device transfers on production paths. Where prior neurosymbolic systems bridge a neural network to a single symbolic backend and keep that backend on the CPU, xlog integrates neural networks, as ordinary predicates, with every reasoning mode through one uniform, transfer-free mechanism. Two device-resident mechanisms make the integration sound and fast. First, a fully GPU-native knowledge-compilation pipeline (provenance → CNF → Decision-DNNF → arithmetic circuit) turns a network's outputs into a differentiable circuit whose forward pass is exact weighted model counting and whose backward pass yields exact gradients; every compiled circuit is proven, by an on-GPU CDCL solver, logically equivalent to its source formula, making circuit correctness a machine-checked certificate rather than an assumption. Second, gradients cross the neural–symbolic boundary zero-copy through DLPack into PyTorch's autograd. We report single-system ablations on one development GPU: circuit caching yields a 2.74× end-to-end training speedup (95% CI [2.29, 3.18]) on MNIST addition, and a worst-case-optimal join subsystem yields a 27.96× geometric-mean speedup over xlog's own binary-join baseline on skewed program-analysis workloads. These are ablations against xlog's own baselines, not head-to-head comparisons; the contribution is the unified, transfer-free neurosymbolic architecture.

1 Introduction

Neurosymbolic systems combine neural perception with symbolic reasoning, but the two halves run on different machines. Deep learning frameworks—PyTorch, JAX—execute dense tensor computation on the GPU through optimized CUDA kernels. Logic engines—Datalog evaluators, probabilistic-logic systems such as ProbLog [1], answer-set solvers—are CPU-bound interpreters and compilers. Every system that couples the two, from DeepProbLog [2] to NeurASP [3], therefore straddles the PCIe bus: each training iteration ships a network's outputs to a CPU logic engine, materializes query results or proofs, and pulls gradients back to update parameters. At scale—millions of ground atoms, thousands of steps—these host–device transfers, not the inference or learning itself, dominate wall-clock time and bandwidth.

The transfer wall is one half of the problem; partial coverage is the other. Existing neurosymbolic frameworks bridge a neural network to a *single* symbolic backend—DeepProbLog to probabilistic logic, NeurASP to answer-set programming—and run that backend on the CPU. GPU logic efforts, conversely, accelerate one paradigm in isolation: deterministic Datalog [4, 5] or SAT [6], with no neural interface. No system makes the *whole* symbolic spectrum—deterministic, probabilistic, and epistemic reasoning—both GPU-resident and differentiably coupled to neural networks.

xlog closes both gaps. It is a GPU-native logic programming language whose compiler and runtime treat a neural network as an ordinary predicate: the network's softmax output becomes a distribution over weighted facts that flow into whichever reasoning mode a program uses, and gradients flow back, all without leaving the device. Three mechanisms make this integration both universal and sound. A *device-resident symbolic substrate* keeps fact stores, deltas, circuits, solver state, and gradient buffers in GPU memory throughout evaluation, so the symbolic half never crosses the bus. *Verified knowledge compilation* turns a network's outputs into a differentiable arithmetic circuit and proves, on-device, that the circuit is logically equivalent to its source formula—so the neural–symbolic bridge is exact and machine-checked, not merely assumed. *Zero-copy differentiable coupling* exports the resulting gradients through DLPack [7] directly into PyTorch's autograd graph. The system is implemented in Rust with custom CUDA kernels organized into a layered crate architecture; host involvement is limited to orchestration, I/O, and compilation.

The principal contributions of this paper are:

- **A universal neurosymbolic integration model.** A neural network is a predicate, and the same uniform, transfer-free mechanism connects it to deterministic, probabilistic, and epistemic reasoning—rather than a bespoke bridge to one backend (Sections 2 and 6).
- **A GPU-resident symbolic substrate.** Seminaive Datalog evaluation with stratified negation

and aggregation runs as custom CUDA relational kernels, extended with a worst-case-optimal join (WCOJ) subsystem that avoids the intermediate-relation blowup of binary-join plans on skewed cyclic queries (a measured 27.96× geometric-mean ablation, Section 4).

- **GPU-native verified knowledge compilation.** A provenance→CNF→D4→circuit pipeline runs entirely on device, and an on-GPU CDCL solver proves each compiled circuit equivalent to its source formula. To our knowledge this is the first end-to-end GPU-resident knowledge-compilation pipeline with on-device equivalence verification, and it is the paper's central technical result (Section 5).
- **End-to-end differentiable training with zero-copy interop.** Compiled circuits are cached across iterations and evaluated as level-parallel forward/backward kernels, with gradients exported zero-copy via DLPack and Arrow; circuit caching yields a measured 2.74× training speedup. Term embeddings and differentiable ILP ride the same device-resident path (Section 6).
- **Full-spectrum reach.** Epistemic reasoning over world views (`know`/`possible` under founded and Gelfond-style semantics) and well-founded semantics execute on the same substrate, reusing the CDCL, join, and circuit machinery rather than a separate engine (Section 7).

The remainder of this paper is organized as follows. Section 2 presents the integration model and system architecture. Section 3 describes the xlog language. Section 4 details the GPU-resident symbolic substrate. Section 5 presents verified knowledge compilation, the central contribution. Section 6 ties the pieces together as end-to-end neurosymbolic learning. Section 7 extends the substrate to epistemic and other reasoning modes. Section 8 reports evaluation results, Section 9 surveys related work, and Section 10 discusses limitations and future work.

2 Integration Model and Architecture

2.1 The Integration Model

In xlog a neural network *is* a predicate. A declaration binds a network to a predicate symbol; evaluating that predicate runs a forward pass, and the network's softmax over a label set becomes a distribution over weighted facts. Those facts enter whichever reasoning mode the program uses—deterministic joins, probabilistic knowledge compilation, or epistemic world-view search—through the same relational interface, and the gradient of a query's value with respect to the network outputs flows back across the same boundary. The integration is *uniform* (one mechanism, independent of backend) and *transfer-free* (the facts, the compiled circuit, and the gradients never leave the GPU). The rest of this section describes the substrate that makes both properties hold: a strict GPU-residency contract, a layered crate architecture, a compilation pipeline, and an extensible IR stack.

2.2 GPU Residency Model

xlog enforces a hard guarantee: all runtime semantic state resides in GPU device memory, or the system returns a deterministic error. There is no silent out-of-core spilling and no implicit CPU fallback; a workload that exceeds the memory budget raises `RESOURCE_EXHAUSTED` with diagnostics rather than degrading transparently. The contract extends to production query paths, which target zero device-to-host (D2H) transfers: the kernel provider accounts for transfers at byte granularity through atomic counters, so a performance-critical section can be bracketed and asserted to download nothing—the check the training path relies on to prove its gradient path stays on-device.

The payoff is bandwidth asymmetry. PCIe 4.0 ×16 delivers roughly 32 GB/s, while GPU HBM provides 1–3 TB/s. A single unnecessary D2H round-trip for a moderate relation (10M tuples at 16 bytes, ≈160 MB) costs about 5 ms—enough to dominate a semi-naive iteration that completes in microseconds on-device. Keeping fact stores, deltas, circuit values, solver state, and gradient buffers exclusively on the GPU eliminates this class of bottleneck; host involvement is restricted to orchestration, I/O, and compilation.

2.3 Crate Architecture

The workspace is organized into five tiers with strict layering: no crate depends on a peer or a higher tier (Figure 1). A leaf *core* crate defines the shared scalar types, the kernel-provider trait, error and memory-budget types, and a bidirectional symbol table for dictionary-encoded strings. Above it, a tier of intermediate representations and device services wraps CUDA via cudarc [8], stages kernel modules (cubin-first with portable PTX fallback), and exposes Arrow and DLPack interoperability. The three core subsystems—the typed language frontend (Section 3), the GPU runtime executor, and the GPU CDCL/local-search solver—compose into integrated pipelines for deterministic, probabilistic, and neural-symbolic execution, which a PyO3 [9] extension module (`pyxlog`) and a command-line binary surface to users.

2.4 Compilation Pipeline

A program is compiled to a GPU-executable plan in five stages. **Parsing** (a PEG parser built with pest) produces

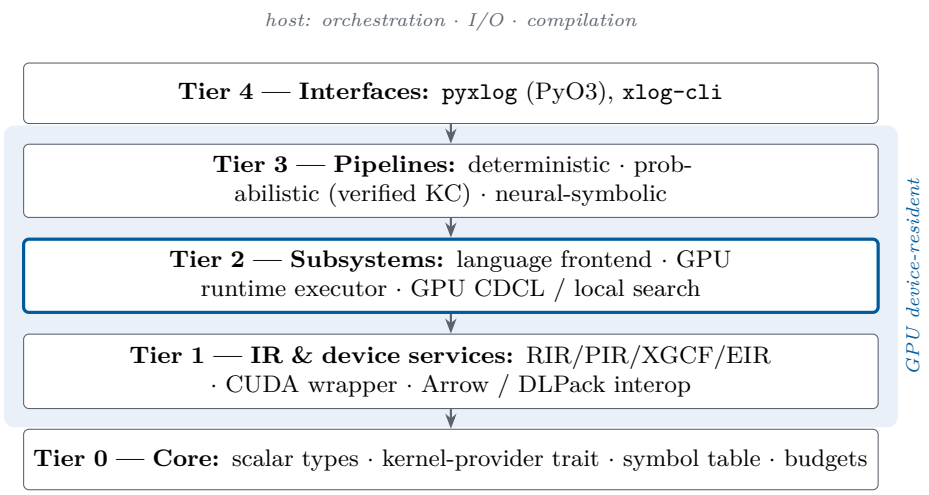

Figure 1: xlog crate hierarchy. Each tier depends only on strictly lower tiers; all runtime semantic state is device-resident, while the host handles only orchestration, I/O, and compilation.

an AST covering the full surface, including probabilistic facts, annotated disjunctions, and neural-predicate declarations. **Stratification** builds a predicate dependency graph with positive, negative, and aggregate edges and computes its strongly connected components (SCCs) with Tarjan's algorithm; any SCC that crosses a negative or aggregate edge is rejected, guaranteeing a unique fixpoint and a deterministic stratum order. **Lowering** translates each rule into a relational-IR tree—body literals become scans, shared variables become join keys, negated literals become anti-joins, and recursive SCCs are wrapped in fixpoint nodes. **Optimization** applies cost-based predicate pushdown and join ordering, promotes eligible multiway rules to the WCOJ subsystem, and rewrites bound recursive queries via magic sets (Section 4). **Plan assembly** emits an execution plan holding SCCs in dependency order, which the runtime interprets by dispatching each node to its CUDA kernel.

2.5 IR Stack

A layered IR stack keeps the backends decoupled; each level has one role.

RIR (Relational IR) is the algebraic tree—`Scan`, `Filter`, `Project`, `Join`, `GroupBy`, `Union`, `Distinct`, `Diff`, `Fixpoint`, and the worst-case-optimal `MultiWayJoin`/`ChainJoin`—carrying cardinality, skew, and delta-vs-full metadata. Every backend consumes it.

PIR (Provenance IR) is the weighted Boolean formula (`Lit`, `NegLit`, `And`, `Or`, `Decision`) derived from provenance tracking over a probabilistic program, capturing its structure without execution order.

XGCF (xlog GPU Circuit Format) is the compiled, device-resident form: a levelized DAG whose level offsets let every node at one topological level evaluate in a single kernel launch, with forward (log-space weighted model counting) and backward (adjoint) passes.

EIR (Epistemic IR) preserves modal operators for world-view reasoning and lowers to a dedicated GPU execution plan rather than ordinary relational algebra (Section 7).

3 The xlog Language

This section describes the xlog surface: types, expressions, user-defined functions, modules, and stratified aggregation. Probabilistic facts (`p::f.`) and neural predicate declarations (`nn/k`) are language-level constructs covered in Sections 5 and 6.

3.1 Design Principles

Three principles shape the surface, each a deliberate enabler of GPU execution rather than a concession. **Typed predicates:** every predicate has a declared signature over a closed set of scalar types (`u32`, `u64`, `i32`, `i64`, `f32`, `f64`, `bool`, `symbol`), so argument mismatches are caught before any kernel is generated and allocation stays bounded and columnar. **Stratified semantics:** negation, aggregation, and recursion interact through an SCC analysis on the predicate dependency graph, and programs whose SCCs cross negation or aggregation are rejected at compile time, guaranteeing a unique fixpoint computable by a deterministic semi-naive schedule. **One language for many paradigms:** probabilistic facts, annotated disjunctions, neural predicates, and SAT constraints share the syntactic core of deterministic Datalog; parsing, dependency analysis, and most lowering are shared, while backend-specific passes refine the plan. A language admitting unbounded terms or recursion through negation would forfeit exactly the allocation and fixpoint guarantees the GPU runtime depends on.

A minimal program exhibits the core surface:

```
// The minimal xlog program: typed predicates, facts, a recursive rule, a query.
pred edge(u32, u32).
pred reach(u32, u32).

edge(1, 2). edge(2, 3). edge(3, 4).

reach(X, Y) :-edge(X, Y).
reach(X, Z) :-reach(X, Y), edge(Y, Z).

?-reach(1, N).
```

Listing 1: Minimal xlog program: typed predicates, facts, a recursive rule, a query.

3.2 Type System

Every predicate is declared with `pred` over the eight scalar types, with `f32`/`f64` following IEEE 754. String literals (`"alice"`) map to `symbol` through a global symbol table, giving the runtime dense integer identifiers while preserving source-level readability; `domain` declarations introduce named aliases. Type consistency is enforced by predicate schemas and lowering-time checks: variable occurrences must agree with declared signatures, constants are validated against their destination columns, and arithmetic expressions preserve scalar types.

```
pred node(u32, symbol).
pred connected(u32, u32).
pred bridge(u32, u32).

node(1, "alice"). node(2, "bob").
connected(1, 2).

bridge(A, B) :-connected(A, B), node(A, _), node(B, _).
```

Listing 2: A well-typed xlog program: typed bridge predicate.

Declaring `bridge` so that its head argument is constrained at two incompatible types (e.g. `symbol` in the head but `u32` through `connected`) rejects the program at compile time.

3.3 Expressions, Arithmetic, and User-Defined Functions

Rule bodies may contain arithmetic, comparisons, and calls. Arithmetic bindings use the `is` operator (`D is X + Y`); the right-hand side supports `+ - * / %`, the built-ins `abs`, `min`, `max`, `pow`, `cast`, and user-defined functions. Comparisons compile to `Filter` nodes and push below joins where profitable. Float comparison uses a total ordering—`NaN` and signed zeros yield deterministic results rather than contaminating downstream aggregations.

```
pred point(u32, f64, f64).
pred close(u32, u32).

point(1, 0.0, 0.0). point(2, 0.5, 0.5). point(3, 5.0, 5.0).

close(A, B) :-
  point(A, Xa, Ya),
  point(B, Xb, Yb),
  A < B,
  D is (Xa - Xb) * (Xa - Xb) + (Ya - Yb) * (Ya - Yb),
  D < 1.0.
```

Listing 3: Arithmetic in rule bodies: pairwise proximity via squared Euclidean distance.

A user-defined function (UDF) is introduced with `func`, with optional parameter types and a return type after `->`. Arithmetic bodies (optionally with `if/then/else`) inline into the same `Expr` trees as built-in arithmetic and participate in the same predicate-pushdown and expression-level optimization. UDFs are pure scalar functions and do not recurse through relations, so a UDF call in a recursive rule does not enlarge the derivable set—closing part of the gap to classical Datalog without breaking monotonicity.

```
pred raw_score(u32, f64).
pred norm_score(u32, f64).

func clamp01(X: f64) ->f64 =
  if X < 0.0 then 0.0
  else if X > 1.0 then 1.0
  else X.

raw_score(1, -0.3). raw_score(2, 0.7). raw_score(3, 1.4).

norm_score(Id, N) :-raw_score(Id, R), N is clamp01(R).
```

Listing 4: User-defined function: `clamp01` normalizes a raw score into the unit interval.

3.4 Modules and Imports

Programs decompose into `.xlog` modules on a search path. The `use` directive loads a module (`use graph.`) or selectively imports names (`use utils/math::{abs, clamp}.`); a `private` modifier hides declarations from importers. Name resolution runs before type inference: the resolver traverses the import graph, detects cycles, merges symbol tables, and surfaces a single logical program, so imported rules participate in exactly the same SCC analysis and GPU semi-naive evaluation as if written inline.

```
// library: graph_lib.xlog
```

```
pred edge(u32, u32).
pred reach(u32, u32).

edge(1, 2). edge(2, 3). edge(3, 4).

reach(X, Y) :-edge(X, Y).
reach(X, Z) :-reach(X, Y), edge(Y, Z).
```

Listing 5: Module `graph_lib.xlog`: predicate declarations and recursive reach rules.

```
// entry: graph_main.xlog
use graph_lib.

?-reach(1, N).
```

Listing 6: Entry `graph_main.xlog`: imports `graph_lib` and queries `reach`.

3.5 Aggregations and Constraints

Aggregation operators—`count`, `sum`, `min`, `max`, `logsumexp`—appear at rule heads with an explicit aggregation variable and lower to `GroupBy` nodes executed as radix-sort-and-reduce kernels; the stratifier rejects aggregation inside a recursive SCC. Integrity constraints are headless rules (`:- body.`) asserting that `body` is unsatisfiable once evaluation reaches fixpoint; a violation raises a diagnostic. `logsumexp` is included because it is the workhorse aggregation for probabilistic circuits, where log-space numerical stability is essential (Section 5).

```
pred edge(u32, u32).
pred degree(u32, u32).

edge(1, 2). edge(1, 3). edge(1, 4). edge(2, 3).

degree(X, count(Y)) :-edge(X, Y).

:-edge(X, _), not degree(X, _).

?-degree(X, N).
```

Listing 7: Stratified aggregation: per-source out-degree with an integrity constraint.

3.6 Completeness over Finite Domains

The surface extends toward language completeness while preserving GPU-native execution: every construct normalizes into the existing typed AST, RIR, and runtime paths, and forms that cannot be lowered safely are rejected at compile time rather than executed on a hidden CPU interpreter. The accepted extensions are finite typed lists (`[]`, `[H|T]`, the `list<T>` column type), statically-resolvable meta-predicates (`ground`, `functor`, `=..`, source-fact `findall`, `maplist`), explicit stratified negation-as-failure over deterministic relations, magic-set rewriting of bound recursive queries, and finite probabilistic aggregates—each lowering to ordinary relational joins and aggregations. Open-ended generators, dynamic database mutation, unrestricted `call/N`, and runtime-variable predicate names remain outside the contract.

4 GPU-Native Symbolic Substrate

The deterministic Datalog engine is the device-resident core the neural bridge and knowledge compiler build on: it is what keeps the symbolic half of a neurosymbolic program on the GPU. The algorithmic approach—semi-naive fixpoint iteration over stratified programs—is well established; the contribution here is engineering. Every relational operator runs as a custom CUDA kernel, delta relations are maintained on-device, and no host–device transfers occur during evaluation.

4.1 Semi-Naive Evaluation on GPU

The executor processes an execution plan as strata ordered by the dependency analysis of Section 2. Non-recursive SCCs evaluate each rule once, dispatching its RIR tree to the appropriate kernel and merging results per head predicate. Recursive SCCs run semi-naive iteration (Algorithm 1): each rule is re-evaluated through delta-rewritten variants—one per recursive scan occurrence, so a self-join such as `p(X,Y), p(Y,Z)` contributes two variants—and the new tuples are unioned, differenced against the full relation, and deduplicated, all on-device, until no predicate produces new tuples. The profiler records per-operator timing, row counts, and peak memory, feeding the optimizer.

Algorithm 1 GPU semi-naive evaluation of a recursive SCC. All buffers are device-resident.

Require: SCC rules $\mathcal{R}$; relation store S (device-resident)
Ensure: least fixpoint of $\mathcal{R}$ merged into S

for each rule $r \in \mathcal{R}$ **do**
  $\Delta_r \leftarrow \textsc{Eval}(r, S) \setminus S$ ▷ seed delta; set difference on device
repeat
  for each rule $r \in \mathcal{R}$ and each recursive scan i in r **do**
    $T_{r,i} \leftarrow \textsc{Eval}(r[\Delta \text{ at scan } i], S)$ ▷ one delta-variant per self-join occurrence
  for each head predicate p **do**
    $\Delta_p \leftarrow \left(\bigcup_{r,i \text{ for } p} T_{r,i}\right) \setminus S_p$; dedup Δ_p
    $S_p \leftarrow S_p \cup \Delta_p$ ▷ union + sorted dedup on device
until all $\Delta_p = \emptyset$ **or** iteration cap reached
free all Δ buffers

4.2 Relational Kernels

xlog implements its relational algebra as direct CUDA kernels rather than wrappers over cuBLAS, cuSPARSE, or Thrust. A library operator imposes its own allocation, synchronization, and host-staging behavior, which would defeat the zero-transfer guarantee of Section 2; bespoke kernels keep every intermediate buffer on-device. Table 1 summarizes them.

Table 1: Core relational CUDA kernels.

Kernel	Purpose
`join`	Hash join with linked-list collision chains and FNV-1a composite hashing over raw key bytes, so all scalar types share one path; count → scan → materialize variants for inner and left-outer joins.
`sort`	Stable 4-bit radix sort (8 passes over 32-bit keys): per-pass histogram, prefix sum, scatter.
`filter`	Comparison operators for column-vs-constant and column-vs-column, with stream compaction via prefix sum.
`groupby`	Sorted-input grouped aggregation: boundary detection, then per-group Count/Sum/Min/Max/LogSumExp reduction.
`dedup`	Sort-based deduplication and set difference with type-aware equality, including IEEE 754 float handling.
`set_ops`	Union (concat + sort + dedup) and set difference via sorted-array binary search.
`scan`	Blelloch parallel prefix sum—the building block for filter compaction, dedup, and group-by.
`pack`	Multi-column key packing into row-major byte arrays with FNV-1a hashing.

The filter kernel makes a correctness-critical choice for floating point. Equality uses standard IEEE 754 semantics (NaN $\neq$ NaN); ordered comparisons use a total-ordering transform that maps an `f64` bit pattern to an `i64` whose integer order matches the IEEE 754 `totalOrder` predicate ($-$NaN $<$ $-$Inf $<$ negatives $<$ $-0.0 < +0.0 <$ positives $<$ +Inf $<$ +NaN), so Datalog programs produce deterministic results for all floating-point inputs.

4.3 Adaptive Join Planning

The optimizer performs cost-based transformations using runtime statistics. Each plan node is costed along four dimensions—expected rows, coordination cost, peak GPU memory, and host–device transfers—with transfers penalized heavily so the planner prefers device-resident plans even at higher raw compute cost. Predicate pushdown decomposes filters above joins into left-, right-, and cross-predicates; join ordering uses dynamic programming up to ten relations and a greedy heuristic beyond, with selectivities learned from prior executions. The executor can export a statistics snapshot and merge it back before the next compilation, closing the loop between execution and optimization.

4.4 Worst-Case Optimal Joins

Binary-join plans can materialize intermediates asymptotically larger than the final result—the classic pathology for cyclic queries such as triangle enumeration. This is not a corner case for xlog's target workloads: program-analysis queries (points-to, call-graph and reachability inference over code-dependency graphs) are dominated by cyclic, skewed joins where the intermediate blowup governs cost. xlog addresses this with a worst-case-optimal join (WCOJ) subsystem [10–12]. A pure-Rust hypergraph planner analyzes rule bodies for multiway-join eligibility, derives a deterministic variable ordering from cardinality and selectivity statistics, and emits a `MultiWayJoin`/`ChainJoin` node when promotion provably preserves output semantics; otherwise the binary-join plan is retained as a certified fallback. The GPU kernel executes the join via count → device prefix scan → materialize over flat sorted-column storage, with a four-byte metadata D2H total and no count-vector transfer. Coverage spans the triangle, 4-cycles, K-clique templates (K=5–8), recursive/SCC integration, and helper-relation splitting that exposes inner skew to root-level histograms. A default-on skew classifier decides per query whether to dispatch WCOJ: high-skew inputs take the WCOJ path, while uniform or empty inputs fall back to the binary-join chain. Section 8 reports the measured speedup.

4.5 Runtime Optimization

Interactive and long-running deployments—REPL sessions, iterative solver loops, incremental training—repeatedly evaluate similar queries over relations that change little between calls. A stateless executor rebuilds join indices and recomputes shared subplans each time, paying the dominant cost repeatedly. Three optimizations exploit this between-call stability, each with explicit controls and zero added data-plane transfers: common-subexpression elimination caches safe deterministic subplans within an evaluation; adaptive re-optimization adopts a compiler-supplied candidate plan when misplan telemetry crosses a configurable ratio, with output-equivalence checks and rollback; and a persistent hash-index manager—keyed on relation generation, schema, and device—retains built indices across repeated sessions with deterministic LRU eviction (Section 8).

4.6 Reversible Symbols

Datalog programs operate on strings, but GPU kernels operate on fixed-width numeric columns. xlog interns each unique string to a dense, sequential `u32` identifier and

resolves it back in $O(1)$ at output time. Symbol columns then *are* ordinary `u32` columns—join, sort, filter, and dedup kernels operate on them with no special-casing—and the string table stays host-side, since variable-length data is needed only at ingestion and output. The `symbol` type maps to Arrow's `Dictionary(UInt32, Utf8)`, so export to columnar consumers (cuDF, Polars, DuckDB) is zero-copy while preserving the compact internal representation.

5 Verified Knowledge Compilation

Knowledge compilation is the bridge that makes a neural network and a logic program differentiable end-to-end. A network's softmax becomes a distribution over weighted facts; provenance tracking over the program yields a weighted Boolean formula; and that formula compiles into an arithmetic circuit whose forward pass is exact weighted model counting (the query probability) and whose backward pass yields exact gradients with respect to the weights—hence with respect to the network outputs. Probabilistic systems such as ProbLog [1] follow this compile-once/evaluate-many model on the CPU, so a neural pairing like DeepProbLog [2] ping-pongs circuit values and gradients across the PCIe bus every iteration. xlog runs the entire pipeline on the GPU—and, crucially, *proves the compiled circuit correct on-device*, so the bridge is sound rather than assumed.

5.1 The Compilation Pipeline

The pipeline transforms a probabilistic program into a GPU-resident arithmetic circuit (the XGCF format) through five stages (Figure 2); each is one or more CUDA kernel launches, with all intermediate state device-resident.

Provenance extraction. Provenance over deterministic evaluation produces a weighted Boolean formula. An on-device interner hash-conses node batches through an atomic GPU hash table, deduplicating structurally identical subformulas; this is essential because grounding can produce exponentially many redundant subformulas, and interning on the host would force the uninterned graph into host memory—reintroducing the very transfer the pipeline exists to avoid.

CNF encoding. A Tseitin [13] transformation lowers the formula to CNF on the GPU, using parallel prefix sums to assign compact, gap-free variable IDs and emit clauses into a device-resident compressed-sparse-row structure.

D4 compilation. The CNF compiles into a Decision-DNNF circuit via a GPU-native variant of the D4 algorithm [14]: a BFS frontier exposes parallelism, per-block DFS workers perform component detection, decision-variable selection by VSIDS-like activity [15], and recursive decomposition. A smoothing pass forces every branch of an OR/decision node to mention the same variable set (required for correct weighted model counting), and the result is levelized so each topological level evaluates in one kernel launch.

Verification and evaluation are described next.

5.2 Verification: a Machine-Checked Certificate

Rather than trust the compiler, xlog proves the compiled circuit C equivalent to its source formula φ (Algorithm 2). It solves $\varphi \wedge \neg C$ and $C \wedge \neg\varphi$ on a GPU CDCL solver; both must be unsatisfiable for $C \equiv \varphi$. This is a complete formal proof, not a sampling check: UNSAT results carry a resolution proof checked on-device, and the solver never reads its status back to the host—a wrong answer triggers a device-side trap rather than silently propagating. Circuit correctness thus becomes a machine-checked certificate, which matters because a silently miscompiled circuit would corrupt every downstream probability and gradient, and hence every parameter update. The same device-resident CDCL substrate is reused for SAT/MaxSAT queries and for epistemic candidate validation (Section 7).

Algorithm 2 GPU-native verified knowledge compilation. The circuit is accepted only if proven equivalent to its source formula on-device.

Require: weighted Boolean formula φ from provenance (device-resident)
Ensure: verified arithmetic circuit C (XGCF), or a device-side trap

$\varphi \leftarrow \textsc{InternHashCons}(\varphi)$ ▷ dedup subformulas via atomic GPU hash table
$\Phi \leftarrow \textsc{TseitinCNF}(\varphi)$ ▷ prefix-sum variable IDs; CSR clauses
$C \leftarrow \mathrm{D4}(\Phi)$ ▷ BFS frontier + per-block DFS; VSIDS decisions; smooth; levelize
$u_1 \leftarrow \mathrm{CDCL}(\varphi \wedge \neg C)$; $u_2 \leftarrow \mathrm{CDCL}(C \wedge \neg\varphi)$
if $u_1 \neq \textsc{Unsat}$ **or** $u_2 \neq \textsc{Unsat}$ **then**
    device-side trap ▷ miscompilation never reaches host or downstream
check resolution proofs of u_1, u_2 on-device
return C ▷ certificate: $C \equiv \varphi$, machine-checked

The verified circuit supports a level-parallel forward pass (log-space weighted model counting, one kernel per level, yielding $\log Z$) and a reverse-order backward pass that accumulates per-variable gradients. Both leave their results in GPU-resident buffers consumable directly by PyTorch via DLPack (Section 6).

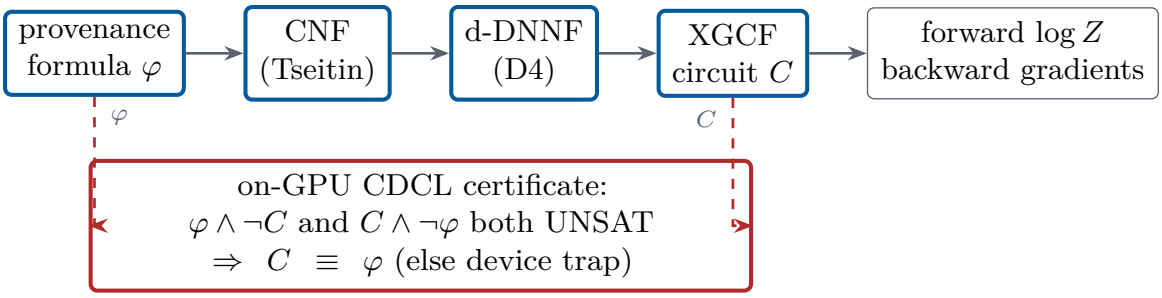

Figure 2: Verified knowledge-compilation pipeline. All stages execute as CUDA kernels on device-resident data; before the circuit is used, an on-GPU CDCL solver proves it equivalent to its source formula.

5.3 Circuit Caching

D4 compilation is the most expensive stage—on the order of a minute for a cold start on the MNIST-addition benchmark—and a naive implementation pays it every epoch. The key observation is that the circuit *structure* depends on the propositional structure of the grounded program and the evidence pattern, but *not* on the numerical weights. When neural parameters change between iterations, the weights on probabilistic facts change while the Boolean formula and its compiled circuit stay identical. The compiled circuit is therefore cached, keyed by a structural hash of the provenance graph and evidence configuration; subsequent iterations update only the weight buffers and run the forward/backward kernels. On MNIST addition this amortization yields a 2.74× end-to-end training speedup (Section 8).

5.4 Monte Carlo Fallback

When exact compilation is infeasible—the ground formula is too large for D4 within the GPU memory budget, or the user accepts approximate results—xlog provides a GPU-parallel Monte Carlo engine. It draws values for all probabilistic facts on the GPU and evaluates the deterministic program in each sampled world, using rejection sampling or, when every evidence atom is a Bernoulli fact, evidence clamping that forces observed values and eliminates rejection waste. Both methods report confidence intervals. The sampler is a GPU-resident megakernel that evaluates each world in a single device launch with a bounded device-side fixpoint; unsupported fragments are rejected fail-closed with a typed diagnostic rather than silently falling back to the host.

6 Neurosymbolic Integration

This section is where the three pillars meet. The device-resident substrate (Section 4) keeps the symbolic state on the GPU; verified knowledge compilation (Section 5) turns a network's outputs into a sound differentiable circuit; and zero-copy coupling carries gradients back into the neural optimizer without leaving the device. Together they let neural perception and logical reasoning train jointly, end to end, with no host–device transfers in the loop.

6.1 Neural Predicates

In xlog a neural network *is* a predicate. The `nn/4` declaration embeds a network directly into the program:

```
nn(network_name, [input_vars], output_var, [↩
    ↪ output_labels]) ::predicate(args).
```

Listing 8: Neural predicate declaration (`nn/4`).

This is not a foreign-function escape hatch: it states that evaluating `predicate(args)` requires a forward pass through the named network, whose softmax over the label set becomes a distribution over probabilistic facts feeding the knowledge-compilation pipeline of Section 5. On the Python side a PyTorch module is registered against the predicate:

```
program.register_network("mnist_net", net, ↩
    ↪ optimizer)
```

Listing 9: Registering a PyTorch module against a neural predicate.

The dataflow runs in two directions. **Forward:** when evaluation reaches an `nn/4`-backed predicate, the registered module runs, and its softmax vector is decomposed into weighted probabilistic facts—one per label—fed into the PIR/CNF/D4/XGCF pipeline. Because circuit structure depends on the program, not the weights, the compiled circuit is cached and only the leaf weights are updated on later iterations. **Backward:** after the circuit computes the forward log-probability and backward gradients via level-parallel kernels, the per-leaf gradient buffers are exported as DLPack tensors and injected into PyTorch's autograd graph, and the optimizer updates the network as usual. The GIL is released during GPU work, so CUDA execution does not block the interpreter.

6.2 End-to-End Training

The canonical example is MNIST addition (the DeepProbLog benchmark): given two digit images, predict their sum, with supervision only on sums—the network never sees individual digit labels. The entire reasoning structure is two lines:

```
nn(mnist_net, [X], Y, [0,1,2,3,4,5,6,7,8,9]) ::↩
    ↪ digit(X, Y).
addition(X, Y, Z) :-digit(X, D1), digit(Y, D2), Z↩
    ↪  is D1 + D2.
```

Listing 10: MNIST addition: a CNN-backed digit predicate and an addition rule.

The `nn/4` declaration connects the CNN to `digit/2`; the addition rule computes every consistent decomposition of a sum through probabilistic marginalization. Each query `addition(i, j, s)` trains by minimizing $-\log P(\texttt{addition}(i, j, s))$ under the compiled circuit. The driver compiles the program, registers the network, and runs the loop, which batches queries sharing a circuit template and runs forward/backward over the cached circuit:

```
program = pyxlog.Program.compile(SRC)
program.register_network("mnist_net", net, ↩
    ↪ optimizer)
program.add_tensor_source("train", train_images)
history = pyxlog.train_model_tensor(program, ↩
    ↪ queries,
                                    epochs=epochs, ↩
                                        ↪ batch_size↩
                                        ↪ =↩
                                        ↪ batch_size↩
                                        ↪ )
```

Listing 11: Python driver: compile, register, train.

The decisive performance property (Figure 3) is that compilation and verification happen once, on the first forward pass; every later iteration reuses the cached circuit, executing only the weight update and the level-parallel forward/backward kernels. This is the mechanism behind the 2.74× training speedup; the loss reduction across seeds confirms that gradients flow from circuit evaluation through the logic program back to the CNN, and the full timing breakdown is reported in Section 8.

6.3 Zero-Copy Interoperability

A GPU-native engine is only useful if results reach the frameworks researchers already use—without copies that would reintroduce the transfer wall. xlog exposes its device-resident state through four standard interfaces.

DLPack. Each relation column or gradient buffer becomes a managed DLPack [7] tensor whose GPU memory is reference-counted and freed only when every consumer releases it; on import, xlog validates device, dtype, contiguity, and alignment, optionally against an expected schema. In Python the protocol surfaces as a capsule, so `torch.from_dlpack` yields a live CUDA tensor backed by the very memory xlog wrote.

Arrow. For columnar tools (Polars, DuckDB, cuDF), relations export through the Arrow [16] C Device Data Interface; symbol columns export as Arrow dictionary arrays, so a consumer reads symbolic columns as native string dictionaries while xlog keeps the compact integer representation.

Python bindings. The `pyxlog` extension (PyO3 [9]) exposes compilation, evaluation, training, and result extraction; tensor-valued results are returned as DLPack capsules, and long-running GPU operations release the GIL so data-loader threads proceed concurrently.

PyTorch autograd. The circuit behaves as a differentiable function inside PyTorch: the network's grad-enabled forward feeds the circuit, the scalar loss is exported via DLPack and re-imported, and a batched path stacks inputs into one forward pass and runs a single backward—so gradients flow from the logic layer back to `nn.Module` parameters with no custom autograd `Function`.

6.4 Term Embeddings and Differentiable ILP

Two further paths ride the same device-resident bridge. **Term embeddings** attach dense, optionally trainable vectors to logical symbols: where `nn/4` maps perception to discrete facts, embeddings give logical entities a continuous representation that participates in neural computation, with gradients flowing through the embedding lookup—enabling, for example, knowledge-graph entity representations trained jointly with neural perception. **Differentiable ILP** learns first-order rules rather than network weights: candidate rules are represented as sparse bitmasks over the ground-atom space, credit assignment runs entirely on-device via scatter–gather, and a six-gate promotion pipeline (convergence, novel-rate, regression, held-out F1, ambiguity, typed-schema checks) controls which rules are committed. This sparse, GPU-resident formulation avoids the cubic blowup of dense rule materialization; it is a beta subsystem (Section 10).

7 Epistemic and Other Reasoning Modes

The substrate generalizes beyond what is *true* or *probable*. This section covers three further modes that reuse the same CUDA join, circuit, and CDCL machinery—epistemic reasoning over world views, probabilistic aggregates, and well-founded semantics—broadening what

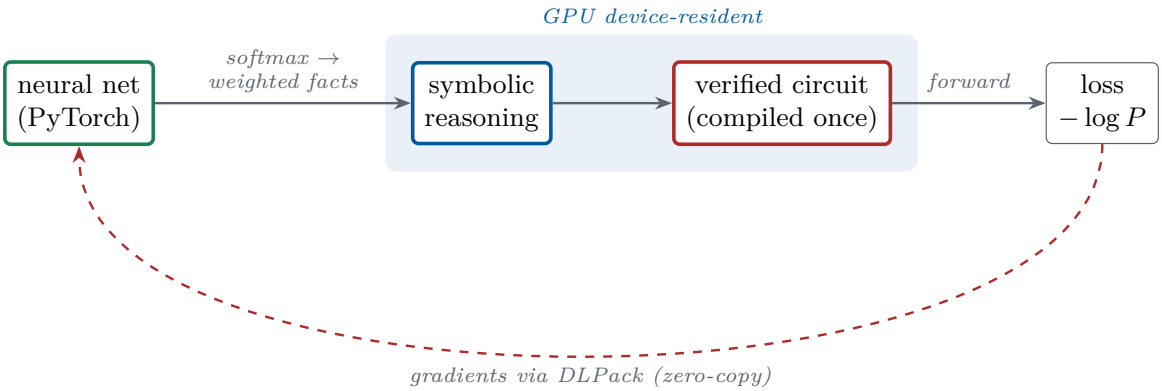

Figure 3: One end-to-end neurosymbolic iteration, exhibiting the three pillars: a device-resident symbolic substrate (the shaded region never leaves the GPU), verified knowledge compilation (the circuit is compiled and proven correct once, then reused), and zero-copy differentiable coupling (gradients return through DLPack into PyTorch's autograd graph).

a neural network can be integrated with while keeping every mode device-resident.

7.1 Modal Surface and the Epistemic IR

Epistemic logic programming asks what is *known* or *possible* across the multiple stable models a program admits—the natural setting for introspection and reasoning under incomplete knowledge. A program may use four modal literals over a predicate—`know`, `possible`, `not know`, `not possible`. Rather than rewrite them away at parse time, the compiler preserves them in an Epistemic IR (EIR) between the AST and GPU lowering, so the world-view semantics are resolved by a dedicated plan rather than smuggled into relational algebra. The accepted surface spans finite nested modal chains, epistemic integrity constraints (which prune candidate world views on the GPU), rules that couple several epistemic predicates, same-name multi-arity modal predicates, and recursive epistemic execution.

7.2 World-View Semantics

A world view is a non-empty set of stable models. `know p` holds when `p` appears in every model of the world view; `possible p` holds when it appears in at least one. Two semantics are selectable: the default, FAEEL (Founded Autoepistemic Equilibrium Logic), rejects circular epistemic support unless it is independently founded, so a self-supporting rule such as `p :- possible p.` fails closed; a Gelfond-1991-style compatibility mode admits such self-support for legacy specifications. Non-epistemic programs evaluate identically under both.

7.3 GPU-Native Execution

Accepted programs lower from EIR to a GPU plan following a Generate–Propagate–Test schedule whose phases run as CUDA kernels: *generate* enumerates bounded candidate assumptions as device bitsets; *propagate* prunes candidates that contradict known or rejected facts; and *test* validates survivors against the program's world views, checking stable-model tuple membership on-device per arity. The reduced ordinary part of each rule reuses the optimizer, WCOJ planner, and helper-splitting machinery of Section 4, and *epistemic splitting* decomposes a program into independent components solved separately and recombined. Candidate validation reuses the on-GPU CDCL solver already built for compilation verification (Section 5)—the same device-resident substrate handles satisfiability, bounded MaxSAT, and assumption-based queries—and accepted world-view evidence feeds the existing GPU-native exact path, so a query can combine epistemic and probabilistic structure without leaving the device. The accepted hot path performs no CPU fallback; a transfer budget tracks that no data-plane host transfers occur within it.

7.4 Probabilistic Aggregates

Finite aggregate queries (`count`, `sum`, `min`, `max`, `logsumexp`) are supported in both exact provenance/PIR inference and Monte Carlo execution, subject to a typed exact-domain cap that rejects domains too large for tractable enumeration. For small finite `count` domains, *aggregate lifting* replaces naive world enumeration with exact cardinality dynamic programming over a compact state set—collapsing, for example, a 2^{17}-outcome enumeration into a few hundred lifted states with identical results—and accepted `count` aggregates dispatch a GPU-native exact evaluator rather than a CPU finite-world shortcut,

keeping the path device-resident.

7.5 Well-Founded Semantics

Programs that violate stratification—such as the classic `p :- not q. q :- not p.`—have no two-valued stable model and are rejected by most Datalog engines. xlog instead assigns three-valued truth (true, false, undefined) via Well-Founded Semantics, computing an alternating fixed point: mark the greatest unfounded set false, derive consequences true, and repeat until stable. For probabilistic programs, true atoms receive normal probability and gradient while false and undefined atoms are assigned probability zero, matching the conservative treatment of non-stratified programs. WFS is invoked automatically when a non-monotone SCC is detected during provenance extraction.

8 Evaluation

This section reports artifact-backed measurements for xlog's deterministic, probabilistic, and neural-symbolic subsystems. Every quantitative claim is tied to a checked-in benchmark artifact. *These are single-system measurements on development hardware: ablations of xlog against xlog's own baselines, not controlled head-to-head comparisons against other engines* (Section 10). Throughput-target envelopes for operations measured only in isolation are maintained as an internal regression contract and are not restated here.

8.1 Methodology

All measurements were collected on an NVIDIA RTX PRO 3000 (Blackwell, 12 GB VRAM, compute capability 12.0). Rust crates are built in release mode; CUDA modules are staged cubin-first with PTX fallback and explicit JIT warm-up; Python components use `pyxlog` with CUDA-enabled PyTorch. The benchmark harness uses Criterion.rs with the settings in Table 2, and GPU measurements include warm-up to amortize kernel-module loading and CUDA context setup.

Table 2: Benchmark harness settings.

Setting	Value
Sample size	10–100 runs (benchmark-dependent)
Warm-up	3 iterations (PTX JIT, memory-pool init)
Significance level	0.10
Noise threshold	0.05 (ignore <5% variance)
Seeding	Deterministic LCG

8.2 Worst-Case Optimal Joins

The WCOJ subsystem (Section 4) is measured against the binary-join baseline on four paper-scale fixtures—call-graph edges, Andersen points-to [17], `ddisasm`-style disassembly [18], and a recursive neural-symbolic mining analog—each with 4.19M input rows. We measure at this scale deliberately: WCOJ speedups depend strongly on input size and skew, and synthetic micro-benchmarks systematically overstate them, so multi-rule recursive workloads at millions of tuples are the honest setting. Both paths must produce identical row sets before timing, and every measurement is a median over 10 runs with coefficient of variation ≤ 0.05.

Figure 4 reports the speedups: a 27.96× geometric mean, with every fixture between 26.6× and 29.6×.

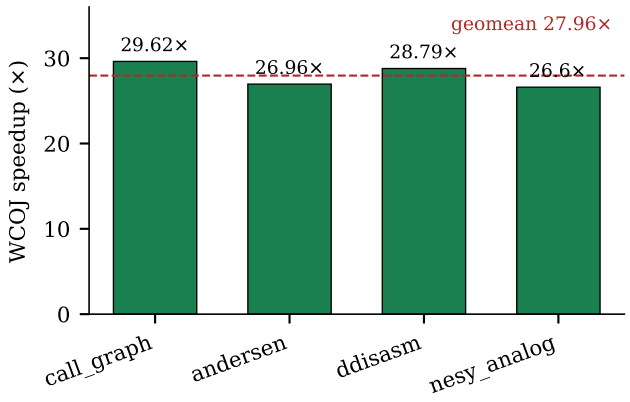

Figure 4: WCOJ vs. binary-join speedup on four paper-scale fixtures (4.19M input rows each). Single-system ablation; the dashed line is the geometric mean.

On uniform or empty inputs the adaptive classifier falls back to the binary-join chain. Peak observed allocation was about 2.3 GB, within the device's 12 GB budget.

8.3 Circuit Caching and Training

Neural-symbolic training is measured on MNIST single-digit addition (512 training images, 5 epochs, batch size 64). Circuit caching (Section 5) amortizes D4 compilation across epochs: a cache ablation over 3 seeds and 3 epochs reduced mean training time from 242.9 s (uncached) to 88.9 s (cached), a 2.74× speedup (95% CI $[2.29, 3.18]$). Table 3 summarizes the full-run timing: the first epoch pays cold-start compilation and verification, while warm epochs run only the cached forward/backward kernels.

8.4 Runtime Optimization

The runtime optimizations (Section 4) are measured on repeated-session fixtures. The persistent hash-index manager records a 3.21× speedup on a build-heavy repeated-session semi-join (cached median 0.08 s vs. uncached 0.26 s) with zero tracked host transfers, and

Table 3: MNIST-addition training summary (mean over 3 seeds).

Metric	Value	Notes
First epoch (cold)	71.7 s	D4 compile + CDCL verify
Steady-state epoch (warm)	0.25 s	cached forward/backward path
Total training (5 epochs)	72.6 s	
Per-query cost	0.97 ms	forward + backward
Circuit-cache speedup	2.74×	95% CI [2.29, 3.18]

a profile-gated shared-memory chain scorer records a 5.58× speedup on a chain-hot fixture with a bounded 1.3% regression on the small-input control. Common-subexpression elimination and adaptive re-optimization are validated for output equivalence rather than reported as headline speedups.

8.5 Capability Comparison

Table 4 positions xlog qualitatively against representative systems; each addresses a subset of the design space, and xlog's contribution is unifying them in one GPU-resident runtime. The comparison is qualitative: xlog has not been benchmarked head-to-head against these systems on identical workloads.

8.6 CUDA Certification

The CUDA kernel certification suite covers all GPU kernel operations: 207 tests across 33 categories (25 infrastructure C01–C25 + 8 GPU-specific G01–G08), at 100% pass on the latest full certification, covering 22 kernel modules. The infrastructure categories cover toolchain validation, launch configuration, memory spaces, synchronization, warp-level operations, atomics, floating-point and determinism semantics, and host-device transfer accounting; the GPU-specific categories validate circuit forward/backward evaluation, weight injection, transfer efficiency, circuit caching, PTX robustness, and SAT/CDCL solving. Because the public repository does not rely on hosted GPUs, the suite runs through GPU-backed local release validation; the always-on CI covers formatting, linting, and no-GPU build checks.

9 Related Work

xlog draws on five lines of research—neurosymbolic programming, GPU-accelerated Datalog, probabilistic logic programming, GPU SAT, and differentiable ILP—and is, to our knowledge, the first to make the full symbolic spectrum GPU-resident and uniformly differentiable.

9.1 Neurosymbolic Programming

DeepProbLog [2] replaces selected probabilistic facts with neural-network outputs, enabling end-to-end gradient training of neural predicates within a probabilistic-logic framework; NeurASP [3] integrates network outputs as probability annotations on answer-set programs. Both perform symbolic inference on the CPU and shuttle gradients across the bus. Scallop [19] provides a differentiable Datalog with provenance-semiring reasoning and a relational symbolic representation, and is the closest language-level analogue to xlog; its reasoning core, however, is CPU-based. Lobster [20] is the most directly comparable system: it GPU-accelerates neurosymbolic programs in the Scallop style and shares xlog's premise that the symbolic side belongs on the GPU. xlog is complementary along three axes: it covers the *full* symbolic spectrum (exact knowledge compilation and epistemic reasoning, not only provenance-based differentiable Datalog); it *verifies* each compiled circuit on-device via CDCL, making circuit correctness a machine-checked certificate; and it enforces a strict residency contract with byte-level transfer accounting. We make no head-to-head performance claim against Lobster (Section 10).

9.2 GPU-Accelerated Datalog and Joins

GPUlog [4] ported bottom-up fixed-point computation to CUDA using HISA indexing and reported up to 45× speedups over CPU Datalog engines; VFLog [5] advanced this with a vertically-fused, column-oriented kernel design avoiding intermediate materialization, reporting up to 200× over CPU column engines; mnmgDatalog [21] extends GPU Datalog to multi-node, multi-GPU clusters. Most relevant to xlog's join path, Sun et al. [22] scale worst-case-optimal joins to GPUs. All target deterministic Datalog only: none supports probabilistic semantics, weighted model counting, or gradient computation over derived facts. xlog shares the columnar, kernel-fused execution model and additionally promotes eligible cyclic and multiway rules to a worst-case-optimal join path [10–12], but treats that engine as one component of a substrate that extends through circuit-based inference to neural-symbolic gradient propagation in a single address space.

9.3 Probabilistic Logic Programming

ProbLog2 [1] established the modern pipeline for exact probabilistic inference: ground the program, encode provenance as a propositional formula, compile to a tractable target (d-DNNF or SDD), and evaluate weighted model counts. The compilers (D4 [14], c2d [23]) run as host-side subprocesses. xlog adopts the same compile-once/evaluate-many model but moves the entire pipeline—provenance, Tseitin encoding, D4 compilation, equivalence verification, and forward/backward

Table 4: Capability comparison across neurosymbolic and GPU logic systems.

Dimension	DeepProbLog	Scallop	GPUlog	xlog
Symbolic execution	CPU (Prolog)	CPU (Datalog)	GPU (HISA)	GPU (semi-naive)
Probabilistic inference	Exact (CPU)	Provenance (CPU)	None	Exact d-DNNF (GPU) + MC
Circuit verification	None	None	N/A	On-GPU CDCL certificate
Neural integration	Prolog bridge	Differentiable	None	Zero-copy DLPack, fused backward
Zero-copy ML interop	No	Partial	No	Yes (device-resident tensors)
Epistemic reasoning	No	No	No	Yes (world views, GPU)

evaluation—onto the GPU, so probabilistic inference shares one address space with the neural and deterministic layers.

9.4 GPU SAT

ParaFROST [6] parallelizes CDCL clause-database simplification on the GPU while retaining sequential conflict-driven search on the CPU; FastFourierSAT [24] reformulates SAT as continuous optimization and applies GPU local search, an incomplete solver for satisfiable instances. Both are standalone competition solvers. xlog's GPU CDCL module instead serves as a verification component inside knowledge compilation (Section 5), proving circuit–formula equivalence via two UNSAT proofs and providing the certificates the probabilistic and epistemic backends rely on.

9.5 Differentiable ILP

Differentiable ILP [25] casts first-order rule induction as differentiable optimization, scoring candidate rules by soft forward-chaining; dense materialization over the rule space scales cubically in the number of constants, and implementations remain CPU-based. xlog's dILP subsystem (Section 6) replaces dense materialization with sparse GPU bitmasks and on-device scatter–gather credit assignment, avoiding the cubic blowup while preserving differentiability.

9.6 Logic Programming Languages

xlog inherits ideas from a long line of logic languages but is the first to target GPU execution of the full surface. Prolog [26] offers unification and metaprogramming on a CPU interpreter; Mercury [27] introduced strong typing and modes, compiling to native code; Soufflé [28] is a typed Datalog compiling to C++ for program analysis; and clingo [29] provides answer-set semantics on CPU grounders and solvers. xlog adopts the typed-predicate discipline but targets device kernels, and unifies typed Datalog, probabilistic facts, neural predicates, and epistemic operators in one language compiled entirely to the GPU.

10 Limitations and Future Work

10.1 Current Limitations

NVIDIA GPU required. The entire execution stack is written in CUDA C; there is no OpenCL, Metal, or ROCm backend. This is a direct consequence of the GPU-native design: the persistent-index manager, the recorded-launch discipline, and the in-kernel hash-consing of the compilation pipeline all depend on CUDA-specific primitives (cooperative groups, warp-level operations, unified virtual addressing), and a lowest-common-denominator portability layer would erode the zero-transfer performance model that is the system's reason for existing. Users without an NVIDIA GPU cannot run xlog.

All data must fit in GPU memory. Relations, indices, circuit buffers, and Monte Carlo sample arrays are allocated entirely on-device; there is no out-of-core spilling. A working set exceeding VRAM fails with `RESOURCE_EXHAUSTED` rather than degrading silently. For most Datalog workloads this is not a bottleneck—a 24 GB GPU holds billions of 8-byte tuples—but large knowledge graphs or high-sample-count Monte Carlo inference can reach the ceiling.

Beta and bounded surfaces. The differentiable-ILP subsystem (Section 6) is beta: the search space is restricted to definite programs, convergence is sensitive to learning-rate schedules, and its API may change. The epistemic runtime (Section 7) executes its accepted surface GPU-natively, but the broader solver portfolio (MaxSAT/portfolio services) and the end-to-end probabilistic–epistemic evidence path are still being broadened, and a negated-modal-cycle well-founded path is GPU-backed but still host-orchestrated. Genuine fail-closed boundaries persist by design—raw lowering of modal literals, genuinely cyclic modal coupling with no

founded order, and unbounded tuple-key forms are rejected with typed diagnostics. The Python batch-query helpers are typed but do not yet accept floating-point uploads, a convenience-API gap rather than a core execution limit.

No head-to-head benchmarks. The measurements in Section 8 are single-system ablations against xlog's own baselines. xlog's probabilistic and neurosymbolic pipelines occupy territory similar to DeepProbLog, Scallop, and Lobster, but no controlled comparison on identical programs and datasets exists; the baseline experiments use different dataset sizes and protocols. Claims of speedup relative to those systems remain informal until such benchmarks are published.

10.2 Future Work

Out-of-core execution. A streaming mode would partition relations across host and device, paging tiles through GPU memory in fixpoint-iteration order, removing the single-GPU-memory ceiling at the cost of added transfers.

Multi-GPU partitioned evaluation. Inspired by mnmgDatalog's [21] radix-hash partitioning and GPU-aware all-to-all shuffle, a multi-GPU backend would distribute relations across devices and coordinate joins over NVLink or PCIe.

Deeper epistemic residency. The remaining epistemic work is stronger residency and coverage: a device-resident, no-host-interaction well-founded path, broader non-finite tuple-key forms, and semantic forms beyond the current finite accepted surface.

References

[1] Daan Fierens et al. "Inference and Learning in Probabilistic Logic Programs using Weighted Boolean Formulas". In: *Theory and Practice of Logic Programming* 15.3 (2015), pp. 358–401.

[2] Robin Manhaeve et al. "DeepProbLog: Neural Probabilistic Logic Programming". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2018.

[3] Zhun Yang, Adam Ishay, and Joohyung Lee. "NeurASP: Embracing Neural Networks into Answer Set Programming". In: *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI)*. 2020.

[4] Yihao Sun et al. "Optimizing Datalog for the GPU". In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2025. DOI: 10.1145/3669940.3707274.

[5] Yihao Sun et al. "Column-Oriented Datalog on the GPU". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 39. 14. 2025, pp. 15177–15185. DOI: 10.1609/aaai.v39i14.33665.

[6] Muhammad Osama, Anton Wijs, and Armin Biere. "SAT Solving with GPU Accelerated Inprocessing". In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Vol. 12651. Lecture Notes in Computer Science. Springer, 2021, pp. 133–151. DOI: 10.1007/978-3-030-72016-2_8.

[7] *DLPack: Open In Memory Tensor Structure*. https://github.com/dmlc/dlpack. Accessed 2026.

[8] *cudarc: Safe Rust Wrapper around CUDA*. https://github.com/coreylowman/cudarc. Accessed 2026.

[9] *PyO3: Rust Bindings for Python*. https://pyo3.rs. Accessed 2026.

[10] Hung Q. Ngo et al. "Worst-case Optimal Join Algorithms". In: *Journal of the ACM* 65.3 (2018), 16:1–16:40.

[11] Todd L. Veldhuizen. "Leapfrog Triejoin: A Simple, Worst-Case Optimal Join Algorithm". In: *International Conference on Database Theory (ICDT)*. 2014, pp. 96–106.

[12] Yisu Remy Wang, Max Willsey, and Dan Suciu. "Free Join: Unifying Worst-Case Optimal and Traditional Joins". In: *Proceedings of the ACM on Management of Data (SIGMOD)* 1.2 (2023), 150:1–150:23.

[13] Grigori S. Tseitin. "On the Complexity of Derivation in Propositional Calculus". In: *Studies in Constructive Mathematics and Mathematical Logic* (1968).

[14] Jean-Marie Lagniez and Pierre Marquis. "An Improved Decision-DNNF Compiler". In: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI)*. 2017.

[15] João P. Marques-Silva and Karem A. Sakallah. "GRASP—A New Search Algorithm for Satisfiability". In: *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 1996.

[16] *Apache Arrow: Cross-Language Development Platform for In-Memory Data*. https://arrow.apache.org. Accessed 2026.

[17] Lars Ole Andersen. "Program Analysis and Specialization for the C Programming Language". PhD thesis. University of Copenhagen, 1994.

[18] Antonio Flores-Montoya and Eric Schulte. "Datalog Disassembly". In: *USENIX Security Symposium*. 2020, pp. 1075–1092.

[19] Ziyang Li, Jiani Huang, and Mayur Naik. "Scallop: A Language for Neurosymbolic Programming". In: *Proceedings of the ACM on Programming Languages (PLDI)* 7 (2023), 166:1–166:25.

[20] Paul Biberstein et al. "Lobster: A GPU-Accelerated Framework for Neurosymbolic Programming". In: *Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. arXiv:2503.21937. 2026.

[21] Ahmedur Rahman Shovon et al. "Multi-Node Multi-GPU Datalog". In: *Proceedings of the 39th ACM International Conference on Supercomputing (ICS)*. 2025. DOI: 10.1145/3721145.3730431.

[22] Yihao Sun et al. *Scaling Worst-Case Optimal Datalog to GPUs*. arXiv:2604.20073. 2026.

[23] Adnan Darwiche. "New Advances in Compiling CNF into Decomposable Negation Normal Form". In: *Proceedings of the 16th European Conference on Artificial Intelligence (ECAI)*. 2004.

[24] Yunuo Cen, Zhiwei Zhang, and Xuanyao Fong. *Massively Parallel Continuous Local Search for Hybrid SAT Solving on GPUs*. arXiv:2308.15020. 2023.

[25] Richard Evans and Edward Grefenstette. "Learning Explanatory Rules from Noisy Data". In: *Journal of Artificial Intelligence Research* 61 (2018), pp. 1–64.

[26] Jan Wielemaker et al. "SWI-Prolog". In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 67–96.

[27] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. "The Execution Algorithm of Mercury, an Efficient Purely Declarative Logic Programming Language". In: *Journal of Logic Programming* 29.1-3 (1996), pp. 17–64.

[28] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. "Soufflé: On Synthesis of Program Analyzers". In: *Computer Aided Verification (CAV)*. Springer, 2016, pp. 422–430.

[29] Martin Gebser et al. "Multi-shot ASP Solving with clingo". In: *Theory and Practice of Logic Programming* 19.1 (2019), pp. 27–82.